In [18]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
import warnings
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
warnings.filterwarnings('ignore')
```

In [19]:
```
# for demonstration purposes, i'll be using spaceship titanic datas
train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")
train_df.head()
```

Out[19]:

	PassengerId	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	Ro
0	0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	
1	0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	
2	0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	
3	0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	
4	0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	

In [20]:
```
train_df.columns
```

Out[20]:
```
Index(['PassengerId', 'HomePlanet', 'CryoSleep', 'Cabin', 'Destina
tion', 'Age',
       'VIP', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', '
VRDeck',
       'Name', 'Transported'],
      dtype='str')
```

In [21]:
```
MissingTrainValues = train_df.isna().sum().to_dict()
MissingTestValues = test_df.isna().sum().to_dict()
print(MissingTrainValues)
print(MissingTestValues)
```

```
{'PassengerId': 0, 'HomePlanet': 201, 'CryoSleep': 217, 'Cabin': 19
9, 'Destination': 182, 'Age': 179, 'VIP': 203, 'RoomService': 181, '
FoodCourt': 183, 'ShoppingMall': 208, 'Spa': 183, 'VRDeck': 188, 'Na
me': 200, 'Transported': 0}
{'PassengerId': 0, 'HomePlanet': 87, 'CryoSleep': 93, 'Cabin': 100,
'Destination': 92, 'Age': 91, 'VIP': 93, 'RoomService': 82, 'FoodCou
rt': 106, 'ShoppingMall': 98, 'Spa': 101, 'VRDeck': 80, 'Name': 94}
```

In [22]:
```
CategoricalValues = [
    i for i in train_df.columns
    if train_df[i].dtype in ['object', 'string']
    and i not in ['PassengerId', 'Name','Transported']
]
NumericalValues = [
    i for i in train_df.columns
    if train_df[i].dtype in ['int64', 'float64']
    and i not in ['Transported']
]
TestCategoricalValues = [i for i in test_df.columns if (test_df[i].
TestNumericalValues = [i for i in test_df.columns if test_df[i].dty

print(CategoricalValues)
print(TestCategoricalValues)
print(NumericalValues)
print(TestNumericalValues)
```

```
['HomePlanet', 'CryoSleep', 'Cabin', 'Destination', 'VIP']
['HomePlanet', 'CryoSleep', 'Cabin', 'Destination', 'VIP', 'Name']
['Age', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
['Age', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
```

In [23]:
```
for i in NumericalValues:
    train_df[i] = train_df[i].fillna(train_df[i].mean())

for i in CategoricalValues:
    train_df[i] = train_df[i].fillna(train_df[i].mode()[0])

for i in TestNumericalValues:
    test_df[i] = test_df[i].fillna(test_df[i].mean())

for i in TestCategoricalValues:
    test_df[i] = test_df[i].fillna(test_df[i].mode()[0])
```

In [ ]:
```
X = train_df.drop('Transported', axis=1).copy()
y = train_df['Transported']
TestX = test_df.copy()

for df in [X, TestX]:
    df['Cabin_Deck'] = df['Cabin'].str.split('/').str[0]
    df['Cabin_Side'] = df['Cabin'].str.split('/').str[2]
    df.drop(['PassengerId', 'Name', 'Cabin'], axis=1, inplace=True)
    df['CryoSleep'] = df['CryoSleep'].astype(int)
    df['VIP'] = df['VIP'].astype(int)

X = pd.get_dummies(X, dtype=int)
TestX = pd.get_dummies(TestX, dtype=int)
```

```
TestX = TestX.reindex(columns=X.columns, fill_value=0)
print("Feature shape:", X.shape)
```

```
Feature shape: (8693, 24)
```

In [25]:
```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
TestX = scaler.transform(TestX)
```

In [26]:
```
KNNModel = KNeighborsClassifier()
KNNModel.fit(X_train, y_train)
KNNPred = KNNModel.predict(X_val)
print("KNN accuracy:", accuracy_score(y_val, KNNPred))
```

```
KNN accuracy: 0.7734330074755607
```

In [27]:
```
SVCModel = SVC()
SVCModel.fit(X_train, y_train)
SVCPred = SVCModel.predict(X_val)
print("SVC accuracy:", accuracy_score(y_val, SVCPred))
```

```
SVC accuracy: 0.7924094307073031
```

In [28]:
```
KMeansModel = KMeans(n_clusters=2, random_state=42)
KMeansModel.fit(X_train)
KMeansPred = KMeansModel.predict(X_val)

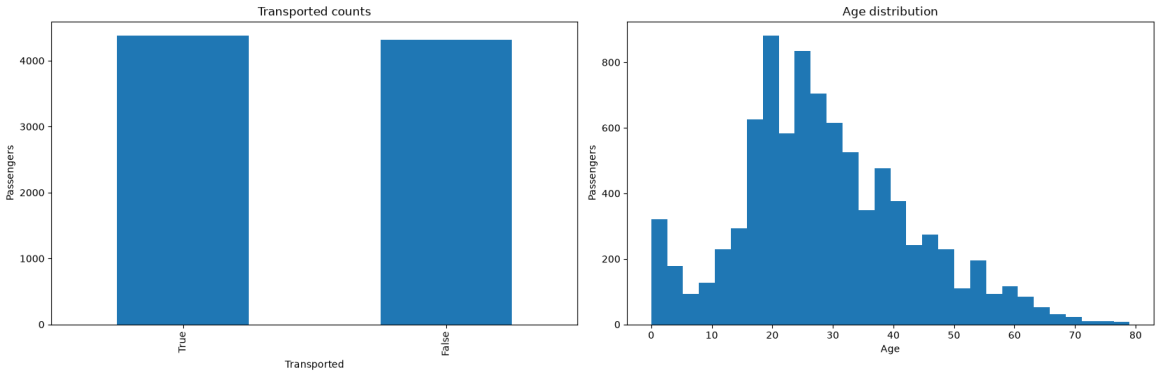
y_val_int = y_val.astype(int).values
acc_direct = accuracy_score(y_val_int, KMeansPred)
acc_flipped = accuracy_score(y_val_int, 1 - KMeansPred)
KMeansAccuracy = max(acc_direct, acc_flipped)
print("KMeans accuracy:", KMeansAccuracy)
```

```
KMeans accuracy: 0.5807935595169638
```

In [29]:
```
plt.figure(figsize=(25,10))
plt.subplot(2, 3, 1)
train_df['Transported'].value_counts().plot(kind='bar')
plt.title('Transported counts')
plt.xlabel('Transported')
plt.ylabel('Passengers')

plt.subplot(2, 3, 2)
plt.hist(train_df['Age'], bins=30)
plt.title('Age distribution')
plt.xlabel('Age')
plt.ylabel('Passengers')

plt.tight_layout()
plt.show()
```



In [30]:
```
ConfusionMatrix = confusion_matrix(y_val, SVCPred)

sns.heatmap(ConfusionMatrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

